

Process Creation

Parent Process (PID 1000)
Code is about to fork()

↓ fork() system call

Kernel creates child process (PID 1001)
with the copy of parent's memory

After fork() returns:

Parent process resumes:

$pid = fork() \rightarrow 1001$
Executes parent-specific code

Child process resumes

$pid = fork() \rightarrow 0$
Executes child-specific code

We can use `exec()` to wipe parent's memory with child's specific code.

Process Termination

- Ideally, we want the parent to finish first & call `wait()` for child's exit status.
- But if the child finishes first, because the parent was still running / busy / blocked, the child's exit status won't be picked up by the parent.
- The child will still occupy the global process list as a PCB, taking up (small) memory.
- The process is said to have turned into a zombie.
- However, it is no longer in the user space. When a process calls `exit()`, all its code, data, stack, heap are released.
- If the parent crashes / terminates without invoking `wait()`, the process becomes an orphan.
- Init process adopts the orphan & calls `wait` on the parent's behalf.

Interprocess Communication (IPC)

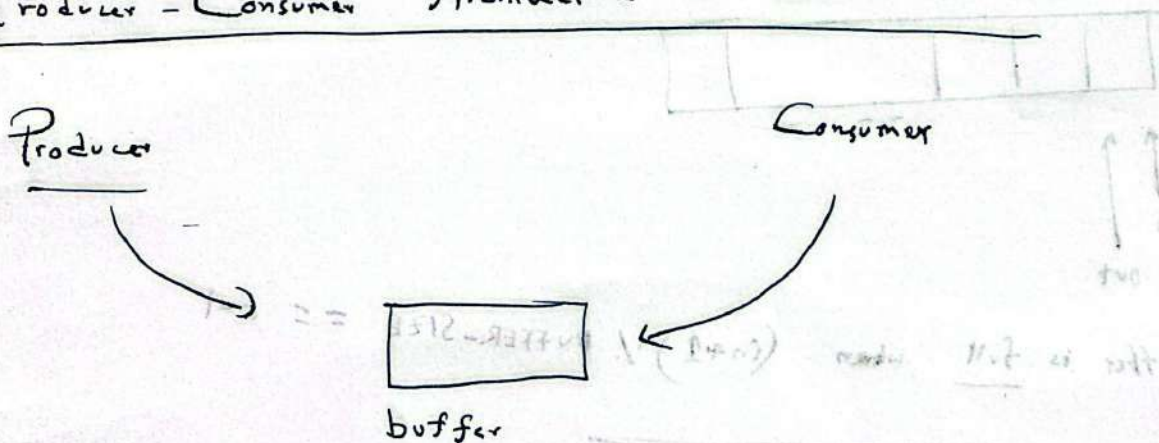
- A process is independent if it does not share data with other processes executing in the system.
- Any process that shares data with other process is a cooperating process.

2 Ways to Share data (for Cooperating processes)

1) Shared Memory : ^{Harder to implement.} Faster. Does not require System Calls

2) Message Passing : Easier to implement, Slower, Requires System Calls.

Producer - Consumer Architecture in Shared Memory

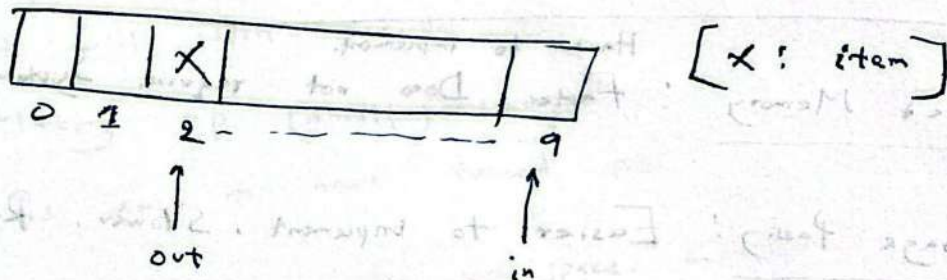


If buffer is bounded, the Producer needs to wait if buffer is full,
the Consumer needs to wait if buffer is empty.

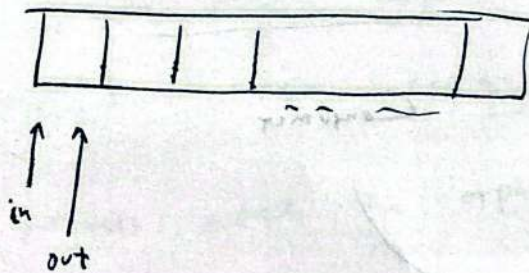
- The shared buffer is implemented as a circular array with 2 pointers, in & out.

in $\rightarrow$ Points to the next free location for the producer to produce an item.

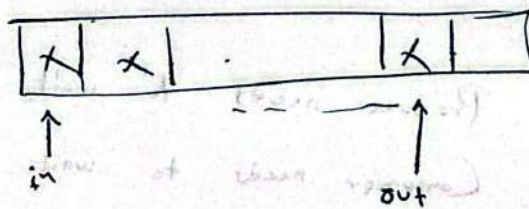
out $\rightarrow$ points to the first full position in the buffer to process ~~an~~ an item (for the consumer)



- The buffer is empty when $in == out$



- The buffer is full when $(in+1) \% \text{BUFFER_SIZE} == out$



When a Producer puts an item:

$buffer[in] \leftarrow next_produced;$

$in \leftarrow (in + 1) \% BUFFER_SIZE;$

When Consumer consumes an item:

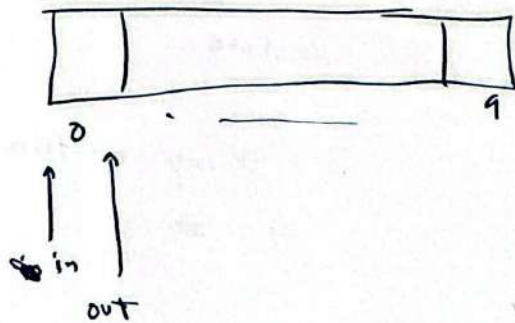
$next_consumed = buffer[out];$

$out \leftarrow (out + 1) \% BUFFER_SIZE;$

Simulation Example

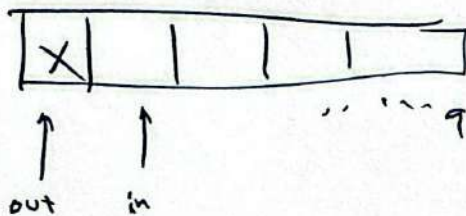
- Initially, buffer is empty

$$in = 0, out = 0$$



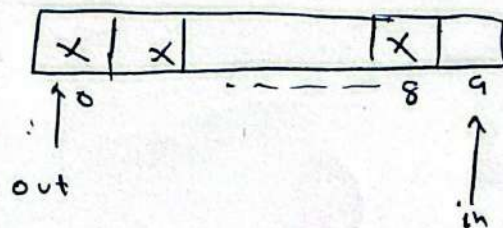
- After 1 item is added

$$in = 1, out = 0$$



- After filling up the buffer with 9 out of 10 items

$$in = 9, out = 0$$



- If the producer added 1 more item, in would update to 0 which would make $in = out$. We have already used this condition to mean empty.